MMA3001 Project – Elijah Khoury – Dataset 3 (Pork Rashers)

Engineering problem

Pork Rashers as well as other meats and goods are distributed all over the world and are packed into plastic trays and sealed airtight. Automated quality inspection in food-packaging is an established and growing idea that can be used to ensure that consumers are receiving safe and quality products. Machine vision and manufacturing automation can be used to develop a reliable automated check that can flag trays early. The problem addressed is: given an RGB image of a packaged tray, detect and localise packaging defects and output a PASS/FAIL decision. This is a manufacturing and automation problem: it combines machine vision, sensing and real-time decision-making in a production-line inspection system, and its performance is limited by engineering constraints such as inference speed (latency per tray), hardware cost and reliability.

Damaged or defected products, especially meat, can result in air getting into the trays, causing the product to spoil faster. A study looked at the effect of leaking on the quality and leak detection of steaks, the results indicated that microbial counts in minced meat steaks increased with increasing leak size at temperatures of 5 degrees Celsius (Eilamo (1995)). Faults that let air in can shorten shelf life and may make the product unsafe, so a defective tray could expose the manufacturer.

Under the Australian Consumer Law, a consumer can seek compensation from a manufacturer who has supplied a product with safety defects if that product has caused loss or damage (ACCC Product Safety, 2024). Manufacturers may look to implement fast, efficient and cost-effective measures to reduce the amount of faulty or defected products that they are sending out.

Visual inspection is a critical part of modern food production, where product quality and safety must be ensured at high speed and with high reliability. Manual inspection is time-consuming and prone to human error, which motivates the use of automated machine vision systems (Augsburg and Ryden,2026).

It is vital to emphasise that computational solution is used as an aid to assist distributors with noticing defects. By itself, the automated check system is not a viable solution, it must be used in conjunction with human verification. The system flags trays that fail for rejection or re-inspection, and trays that pass still receive a human check before distribution.

This project uses the “Pork Rasher Error (Packaging)” dataset (Roboflow Universe, CC by 4.0), containing 1,955 original RGB images (data set splits contain augmented copies roughly 3500) of packaged pork rasher trays with bounding -box annotations for five classes: packaging error, unsealed, wrinkle, twisted-meat and loose-meat. Defects in this project refer to one of five different classes. Unsealed, where the trays film is not fully bounded, leaving a gap. Packaging-error is a visible fault in the film that isn’t a sealing error, such as a tear. Wrinkle refers to a crease in the tray film; Twisted meat is the meat displaced or twisted inside the tray. Loose-meat is meat not held in position correctly.

Conclusions focus on packaging-error and unsealed. The other classes are still trained and reported however cannot draw a conclusion from them, because the validation and test sets contain too few examples to give reliable estimates. As a result, this dataset cannot show how well the model detects those defects.

The main limitations of the scope include images that appear to come from a single fixed camera setup, the test set is small and skewed, the tray level label is derived from the same annotations that train the detector, rather than from an independent inspector, the leakage check only catches augmented copies of the same frame, not near-duplicate frames with different names, and the dataset is a snapshot from one source and may not represent defect variety on other production lines.

Inputs and Outputs –

Top level inputs include the Tray image, settings and the training data. The tray image is an RGB image file of packaged tray. The dataset contains images that are 720 x 540 pixels, 8 -bit colour. The model rescales them whilst keeping the aspect ratio padded to a square input. The format of the RGG image files include .jpg, .jpeg, ,png and .bmp. All of which are acceptable inputs to the model.

The second top level input that solution accepts is the confidence threshold (from 0 to 1) which classes count as a reject, and the minimum number of defects to a full tray (at least 1). The training data for the workflow is the last top-level input. This includes the data set data.yaml, images and the YOLO label files. Formatting of each label line goes as follows class_id, x_center, y_center and width height.

Top level outputs include per-defect detection, this refers to where a defect is located on the package and how sure the model is, so a person can inspect it. The format includes the class name, confidence (0-1), and the box coners (x1, y1 , x2 , y2) in pixels of the original image. This tells the next system or person where the defect is located specifically on that image.

Tray decision refers to the fundamental concept of the model; whether to PASS or FAIL the tray, as a result deciding if it is rejected or if re inspection occurs. Along with the status of the tray, the n_defects (number of defects) and counts per class are included in the tray decision output. The final output is the evaluation results, this is evidence of how well the method works and can be measured using a number of parameters including precision, recall per class average precision, latency, GFLOP's and tray-level precision.

Missing, invalid or unsupported inputs handling can vary based on the case. Missing image files typically show the error 'FileNotFoundError'. Unsupported file type raises a Value Error.

Computational Solution-

A You Only Look Once (YOLO) detector is a real-time computer vision system that identifies and locates objects within digital images using neural networks. It works by looking at the whole image once and predicting boxes, class labels and confidence scores directly. Training adjusts the network's weights to reduce a loss that combines box-position error and classification error.

At inference, overlapping boxes for the same object are merged by non-maximum suppression, and boxes that fall below the confidence threshold are let go. The rule then turns surviving detections into PASS/FAIL. The rule is simple: a tray fails if at least min_defects detections in fail_classes have confidence >= conf_thresh.

The YOLO detector was selected because it can do both localising and classifying the image. Pretrained weights reduce the amount of data needed and the tooling is mature and reproducible. The method is a convolutional neural network, pretrained and fine tuned on the data.

Important assumptions are that defects are visible by the RGB image. If defects are not visible, then the model cannot function as intended. Each box is correctly labelled is another assumption as well as ensuring the distribution of training and test images are the same.

Settings and parameters for the model include the following: pretrained weights (yolov8n.pt and yolov8s.pt), image size (640 or 416), epochs (max 25), early stopping patience (8), batch size (16), random seed (0), deterministic mode, confidence threshold (0.25 default fixed in advance), and which classes cause a FAIL.

The solution was implemented with python using the Ultralytics library on PyTorch, trained on Colab T4 GPU. A small package (data, leakage, train, evaluate, infer, decision) wraps the workflow, with tests and generated docs. The colab pipeline notebook runs the workflow end to end, downloading the data, check and fixing it, training, then comparing, evaluate and profile.

Known limitations include the rare classes (wrinkle, twisted-meat and loose-meat) as they have too few examples to evaluate, small and skewed test set, one source and camera, labels derived from the training annotations, leaked near-duplicate and the blank image false -PASS risk.

Alternative Solutions

Three variants of the same method, a YOLO object detector, were trained and compared: baseline_n_640, s_640 and n_416. All three start from pretrained weights and use the same training data, settings and decision rule, so any difference in results comes from the network size or the input resolution alone.

baseline_n_640 is the baseline. It uses the smallest YOLOv8 network (nano) with images resized to 640 × 640 pixels and gives a cheap reference point for the other two.

s_640 uses the same image size as the baseline but a larger network (YOLOv8 small), with more parameters and therefore more capacity to learn fine differences between defects. The expected cost is slower training, slower inference and higher memory use. The comparison with the baseline tests whether the extra capacity improves accuracy enough to justify that cost.

n_416 uses the same network as the baseline but with images reduced to 416 × 416 pixels. The reduced pixel count lowers the computation per image, which is expected to improve speed, but also reduces the detail available, so small or thin defects may be missed more often. The comparison with the baseline tests how much accuracy is lost for the speed gained. This variant is also an optimisation of the input-resolution setting, which is discussed further in the performance section.

The three variants were compared on the validation split using mAP50-95 for accuracy, and latency, parameter count and weight size for cost . n_416 achieved the highest validation mAP50-95 (0.352, against 0.272 for the baseline and 0.194 for s_640) and the highest recall (0.563) at the lowest latency (10.5 ms), and was selected on that basis. Its precision (0.700) was the lowest of the three, and the differences between variants may be within noise. Contrary to expectation, the larger s_640 was less accurate than the baseline and 47% slower, and the 416 px model was more accurate than the 640 px baseline. The comparison has limits. The validation set contains only 120 images, so small differences in mAP between variants may be within noise, and all three variants use the same underlying method, so the results do not show whether a different type of method (for example a direct tray classifier) would perform better. On the test split the always-FAIL rule scores F1 0.92 and specificity 0. The detector scores F1 0.90 and specificity 0.92. It does not beat the baseline on F1, which is inflated by the 85% FAIL share, but it correctly passes 11 of 12 good trays, which the baseline never does.

Validation

What was validated:

The detector was the first of the validation process. It was validated on whether it was able to find packaging defects and place boxes correctly. Second thing that needed validation was the PASS/FAIL decision built from the detections. The software itself was verified with unit tests (decision rule, error handling, and leakage removal run), which check that the code does what it says. They don’t show that the model performs well.

Validation method and why it’s appropriate:

The test split was not used for training, early stopping or model selection and before evaluation, frames that appeared in more than one split (augmented copies of the same photo) were removed so that the test images are unseen. This estimates performance on new trays from the same source, which is what an inline section step would face. Further checks supplement it: sensitivity testing of the confidence threshold and of the input resolution.

Metrics and acceptance criteria

Detection is report with mAP50, mAP50-95, precision, recall and per-class average precision. The tray decision is reported with precision, recall, specificity and F1, with FAIL as the positive class. Tray-level recall on FAIL was 0.82 (56 of 68; roughly 0.72–0.90), below the 0.90 target, so this criterion was not met at the default threshold of 0.25. Specificity was 0.92 (11 of 12; interval roughly 0.65–0.99), above both the baseline of 0 and the 0.50 target, so this criterion was met. The sweep on the test split was run afterwards for sensitivity analysis only and was not used to choose the threshold.

Influence on model selection:

Validation scores determined early stopping, which epochs weights were kept for each model, and which of the variants was selected. Validation scores are therefore slightly optimistic, and the result comes from the test split, which was evaluated once, for the selected model only.

What the validation cannot establish:

The test set has 80 images, 68 fail and 12 pass trays, so PASS-side results are very uncertain and one tray changes specificity by about 8 percentage points. Wrinkle has no validation or test examples and twisted-meat, and loose-meat have 3-4 boxes each, so nothing reliable can be said about them. All images appear to come from one source and camera, so the results do not show performance on other lines, lighting or packaging. Tray-level ground truth is derived from the same annotations used for training, rather than from an independent inspector.

The leakage check only matches augmented copies by file name, so near-duplicate frames could remain. And inputs that aren't trays, or blank images, would normally return a PASS (no defects found), so the method can't detect camera fault. Furthermore, the recall target was missed and the packaging-error scored low (aP50-95 0.169).

Optimisation

Performance was assessed on accuracy, runtime, memory and input resolution, comparing the baseline (YOLOv8n at 640 px) with a larger model (YOLOv8s at 640 px) and a reduced-resolution model (YOLOv8n at 416 px). Latency was measured with time.perf_counter after 5 warm-up runs, averaged over 50 timed runs on a Tesla T4 GPU, and the GPU time was broken down using the PyTorch profiler. The baseline (YOLOv8n, 640 px) achieved a validation mAP50-95 of 0.272 at a mean latency of 11.7 ms (85.8 FPS), with 3.14 M parameters and 8.75 GFLOPs. YOLOv8s at 640 px was less accurate (mAP50-95 0.194) and 47% slower (17.1 ms, 28.7 GFLOPs, 11.1 M parameters), so its extra capacity did not improve results on this dataset. Reducing the input of the nano model to 416 px cut GFLOPs by 58% (3.70) and gave the highest accuracy (mAP50-95 0.352) at the lowest latency (10.5 ms, 95.0 FPS), so it was selected. The speed-up was much smaller than the FLOP reduction: inference time fell by about 10% (6.84 to 6.14 ms), and the model reached only 7.4% of the T4's peak throughput, which suggests that fixed overheads and memory movement, not arithmetic, limit latency for these small networks. Because the validation set has 120 images, differences in mAP between variants should be treated with caution.

AI Acknowledgment

Which AI tools were used:

Claude was the main AI used during this project. Other AIs used infrequently include copilot and ChatGPT.

What it was used for

- Wrote the initial code: the Python package (data, decision, train, evaluate, infer, later leakage), unit tests, README, licence file and the Colab notebook.
- Scoped the project from the brief and dataset page, and explained the rubric, YOLO, train/validation/test roles and the metrics.
- Helped debug Colab, GitHub and notebook problems.
- Gave feedback on my drafts, and drafted wording for the scope paragraph and the alternatives section.

Why it was used:

AI was used to quickly develop the software structure and tooling, so more of my time could be allocated to the understanding of the intuition behind the process and ideas explored in the model.

How AI was checked:

AI was crossed checked by me. I made sure that I understood what was occurring and how it related to what the project required of me. I did not blindly use it without ensuring I was familiar with what it was producing. Furthermore, I asked it plenty of questions to clarify my understanding and ensure that I was contempt with the work it produced.

Decisions that remained mine:

-The narrowed scope

-Final epoch count

-How to handle leakage

-PASS/FAIL rule

Errors, limitations and unhelpful suggestions:

-A bug in its own code

-Errors present in the code that did not allow for initial setup

-Notebook was rejected on github

-The code guesses the wrong path for best.pt, so evaluation failed.

-The leakage check can't detect near-duplicate frames with different names

How it affected my understanding and workflow:

AI was used throughout this project to further develop my understanding behind the machine learning tools used and needed to create a model able to recognise images. Coming into this project I wasn't very knowledge about the topic and tools like YOLO. AI was able to identify these tools and apply them to the context and from there I was able to learn and understand them.

AI wrote approximately 95% of the code and wrote approximately 30% of the report text.

References

Australian Competition and Consumer Commission. (2024). Product safety laws and liability.

https://www.productsafety.gov.au/business/understand-product-safety-rules/product-safety-laws-and-liability

Augsburg, J. P., & Rydén, K. (2026). *AI-driven machine vision for automated defect classification* [Master's thesis, Chalmers University of Technology]. Chalmers ODR. https://odr.chalmers.se/items/67982e62-1378-4dec-89e7-075df83ff3b7

Eilamo, M., Ahvenainen, R., Hurme, E., Heiniö, R.-L., & Mattila-Sandholm, T. (1995). The effect of package leakage on the shelf-life of modified atmosphere packed minced meat steaks and its detection. *Lebensmittel-Wissenschaft & Technologie, 28*(1), 62–71. https://doi.org/10.1016/S0023-6438(95)80014-X

Pork rasher error (packaging) [Data set]. Roboflow Universe. https://universe.roboflow.com/hello-2aqe0/pork-rasher-error-packaging

Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO* (Version 4) Ultralytics. https://github.com/ultralytics/ultralytics

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 779–788). IEEE. https://doi.org/10.1109/CVPR.2016.91

Brosnan, T., & Sun, D.-W. (2004). Improving quality inspection of food products by computer vision—A review. *Journal of Food Engineering, 61*(1), 3–16. https://doi.org/10.1016/S0260-8774(03)00183-3